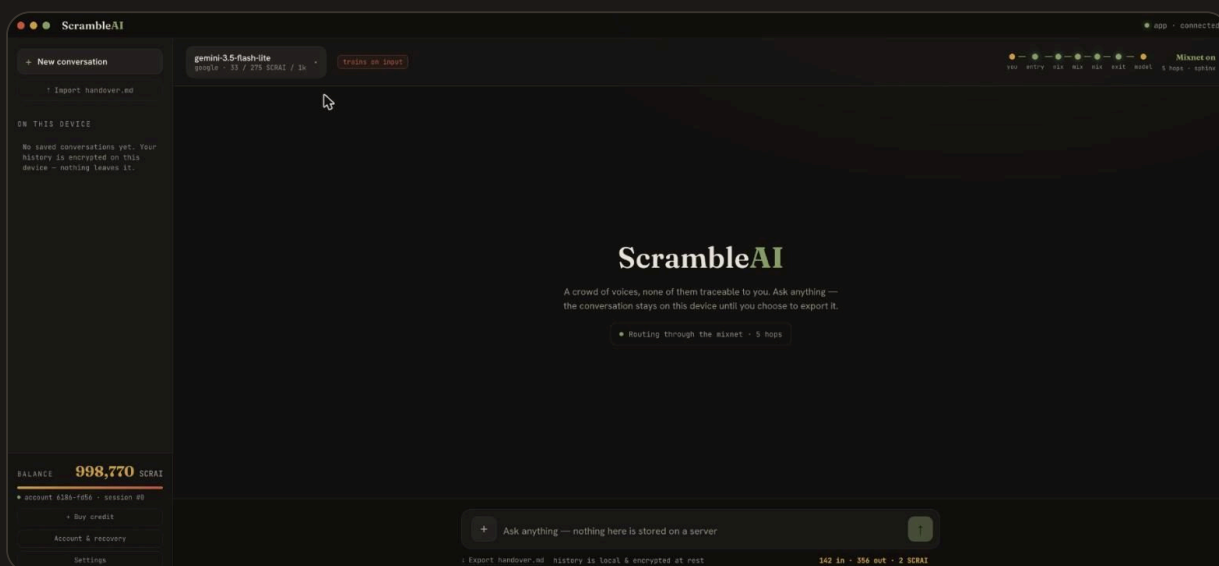


PITCH · AUGUST 2026

ScrambleAI

Anonymous AI for everyone. Every question you ask a chatbot reveals two things: **who** is asking and **what** is being asked. ScrambleAI separates the two — by engineering, not by promise: questions travel through the **Nym mixnet**, payment happens with blind-signed **Coconut credentials** (zk-nym), and your history never leaves the device.

**5 hops**

SPHINX ROUTING

~1.1 s

MIXNET ROUND-TRIP, MEASURED

-60 %

TOKEN COST VIA PRUNING

zk-nym

COCONUT CREDENTIALS

100 % Rust

CLIENT CORE + SERVER

Every screenshot in this document is a real capture of the running MVP — no mockups. Click any image to view it full-size.

© 2026 Hermes Blockchain Ventures — all concepts, code and designs in this document are the intellectual property of Hermes Blockchain Ventures.

Built on open source: Nym SDK & nym-compact-ecash (Apache-2.0) · Tauri (MIT/Apache-2.0) · Rust crate ecosystem incl. tokio, serde, reqwest (MIT/Apache-2.0) · ed25519-dalek (BSD-3-Clause) · SQLite (Public Domain) · BTCPay Server (MIT) · GLiNER & Tesseract (Apache-2.0) · typefaces Fraunces, Hanken Grotesk, JetBrains Mono (SIL OFL 1.1).

Every AI question is a **confession**

Health worries, contract drafts, job applications, relationship trouble — what people confide to an AI is more intimate than any search history. Today all of it lands at a provider under a real-name account, an IP address and a credit card, with permission to train on the content.

Existing "anonymous" routes solve only half the problem: a proxy sees everything itself, a VPN vendor knows its customers, and in the end payment still runs on a card. ScrambleAI is built so that **no single party ever sees "who" and "what" at the same time** — including us. The operator *cannot* be curious.

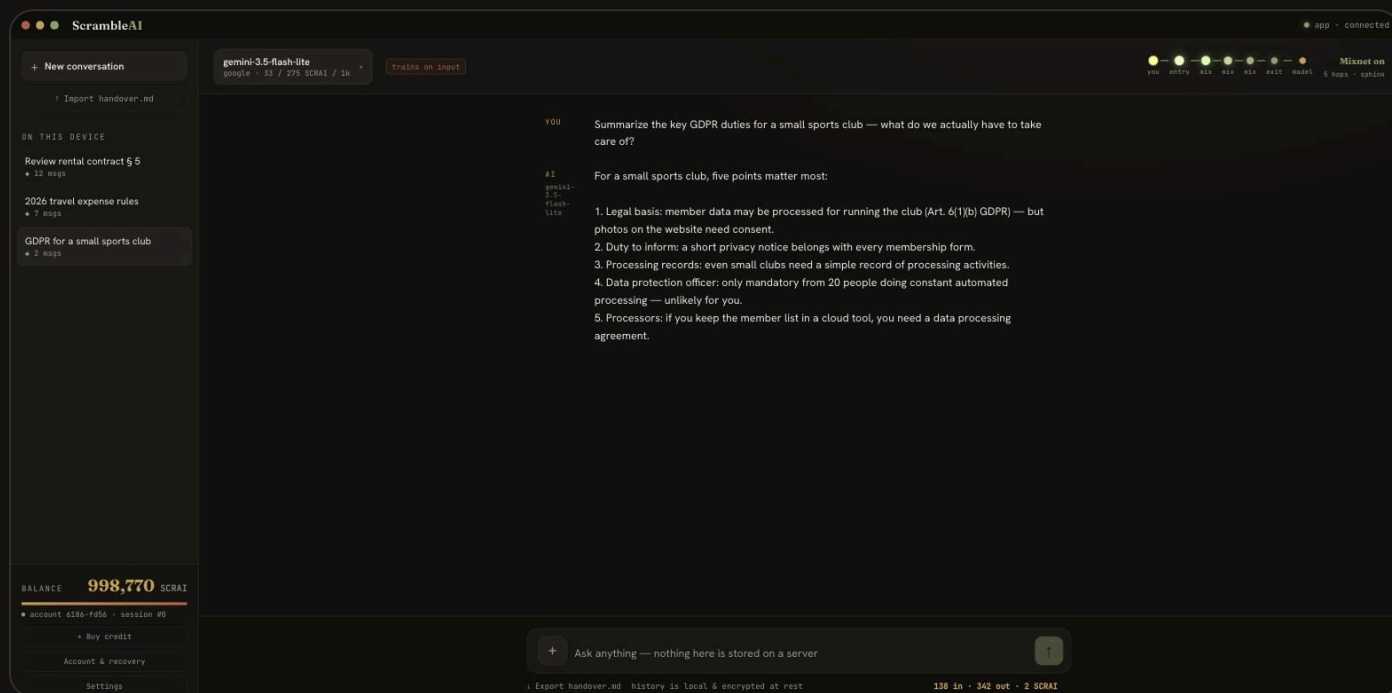
Why this matters for Nym

Mixnet adoption has a chicken-and-egg problem: no traffic without applications, no applications without traffic. ScrambleAI could become **the first commercial consumer application on the Nym mixnet** — a system seller, to put it slightly immodestly: an app people install for the product (private AI, the most confession-heavy workload there is), and that as a side effect generates real Sphinx traffic, real SURB usage, and native demand for NYM on the Nyx chain with every top-up.

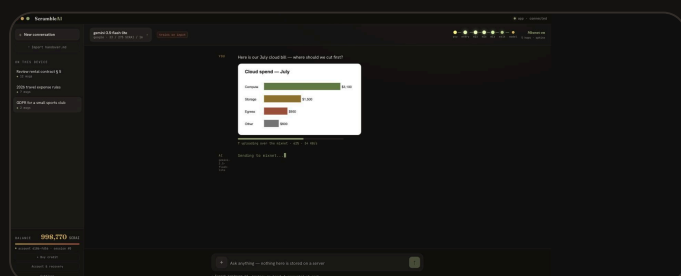
A chat like any other — over a path like no other

ScrambleAI is a desktop app (mobile follows) with an embedded Nym client. Every message is packed into Sphinx packets and routed across five stations: entry gateway, three foreign mix nodes, exit. The server answers through anonymous reply blocks (SURBs) — it never learns a sender address it could remember.

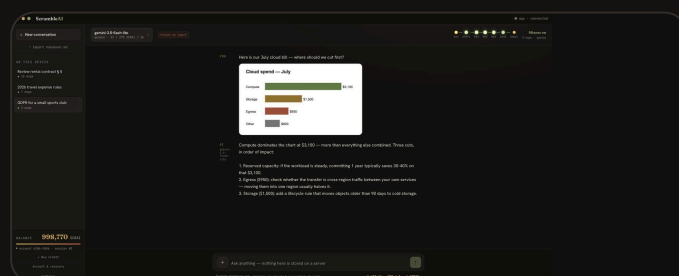
Attachments — images and PDFs — ride the same path: large files are unreliable as one mixnet message, so the app splits them into **acknowledged chunks** with real per-chunk progress, and the server reassembles them only for the request that references them. The conversation itself lives only on the device, encrypted with AES-256-GCM; export and migration use an open, portable format (`handover.md`), passphrase-encrypted on request.



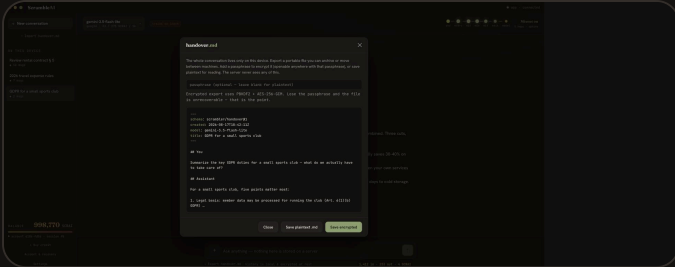
The chat. Top right, the real route: you • entry • mix • mix • mix • exit • model. Left, the encrypted local history. Bottom right, the price of this answer — 138 in • 342 out • 2 SCRAI.



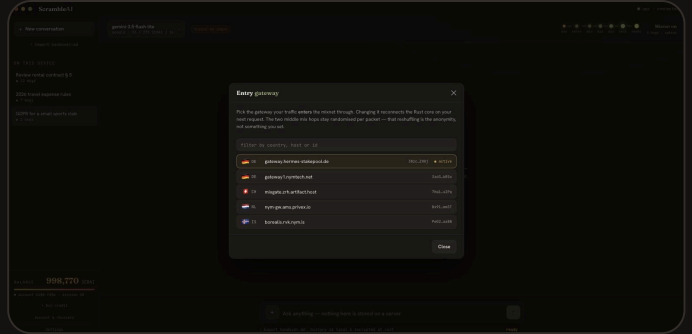
A vision request in flight. The attached image streams up in acked chunks ("uploading over the mixnet • 62% • 34 KB/s") while the status line types out the request's real lifecycle — "Sending to mixnet..." with the live cursor.



The answer, end to end. The model reads the attached chart through the same anonymous path; the footer prices the multimodal request — 1,412 in • 233 out • 4 SCRAI.



Your data, portable. One click exports the conversation as open Markdown – plaintext for reading, or PBKDF2 + AES-256-GCM encrypted for moving between machines.



You choose where you enter. The entry gateway is selectable by country and host; the middle mix hops stay re-randomised per packet – that reshuffling *is* the anonymity.

The app watches out **before** anything leaves the device

Anonymous routing is worthless if your IBAN sits in the prompt. So ScrambleAI checks every message and every attachment **entirely on-device** — none of it is ever sent, logged or stored.

Text scan

Pattern checks for IBANs, card numbers, API keys, passwords, recovery phrases, phone numbers.

Semantic scan (local model)

A GLiNER model runs natively inside the app and catches open classes — names, addresses, employers — zero-shot, no cloud.

File scan with OCR

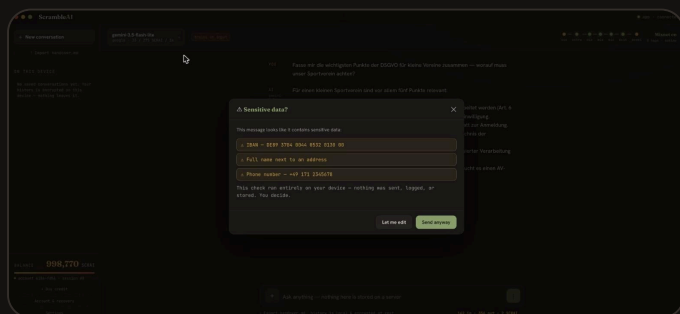
Images and PDFs are read locally (Apple Vision natively, Tesseract-WASM as fallback) and pushed through the same checks. Findings can be redacted right in the image.

Manual redaction mode

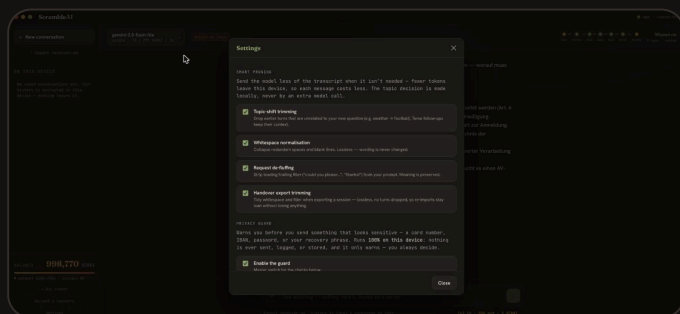
Detection boxes are a starting point, not the authority: dismiss false positives with a click, and drag your own black boxes across anything the scanner missed. The bars are burned into the image before it leaves.

Smart pruning

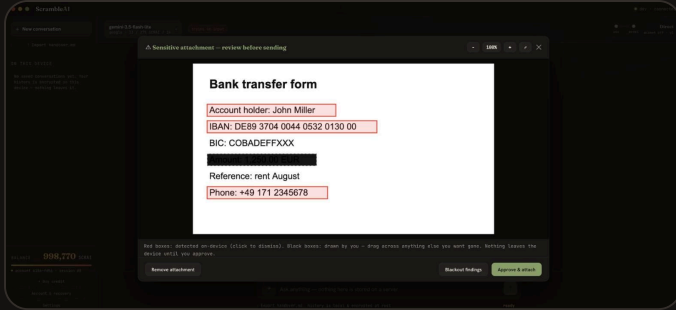
Before sending, the transcript is trimmed lexically to what this question needs — a measured –60 % tokens, without an extra model call.



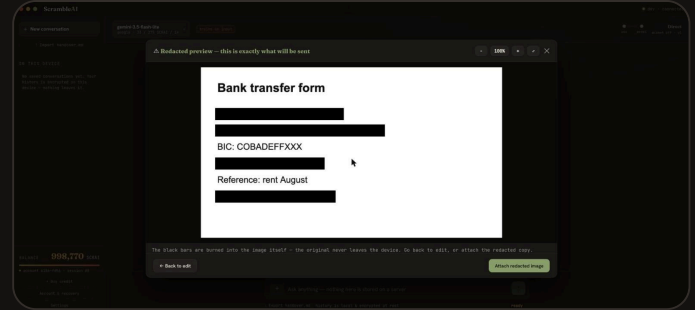
Privacy guard. Warns about an IBAN, a name next to an address, a phone number – "nothing was sent, logged, or stored. You decide."



Settings. Every check can be toggled; the topic decision is made locally, never via a model call.



Attachment review, edit mode. OCR reads the form on-device; the guard flags name, IBAN and phone (red). The black box over the amount was drawn by hand – the manual mode for everything the scanner can't know is sensitive.



The redacted result. Preview before sending: the bars are burned into the image copy itself – the original never leaves the device.

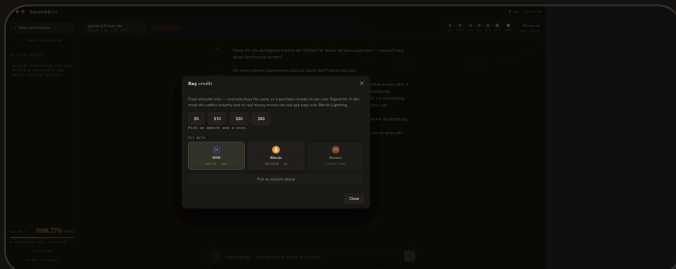
Credit that **detaches** from its buyer

The account is a 24-word phrase (BIP39) — no name, no e-mail. Purchases come in fixed tiers (\$5/10/20/50) so every purchase looks like every other. Then the decisive step happens: the credit is **decoupled** from the account.

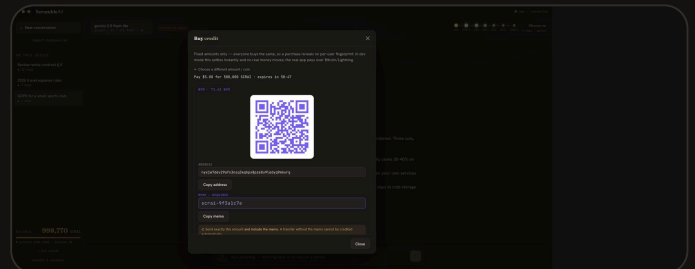
The money's path

- 1 · **Buy.** An invoice over Bitcoin/Lightning (BTCPay) or native NYM on the Nyx chain; Monero follows. The account signs the order — here, linkage is still unavoidable, and honestly so.
- 2 · **Blind withdrawal.** The paid credit is exchanged for **Coconut credentials** (zk-nym / nym-compact-ecash, BLS12-381): the server signs ticketbooks whose contents it *cannot read*. The account→coins link dies cryptographically at this step.
- 3 · **Spend.** The app redeems coins into session credit time-decoupled, in uniform ~\$1 chunks. Every message is Ed25519-signed, its worst-case price reserved up front and settled exactly after the answer — if the provider fails, it costs nothing.

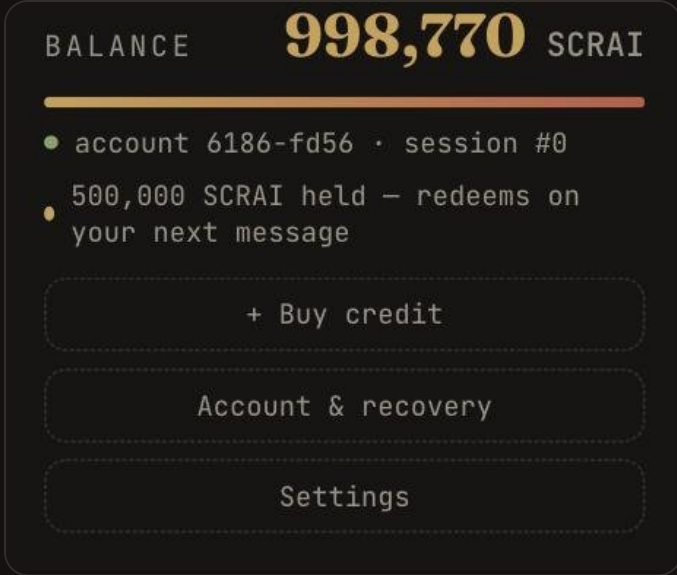
Double-spends are accepted offline and detected by a quorum; sanctions happen **only on cryptographic proof** (two conflicting payments that mathematically expose the key) — never on suspicion.



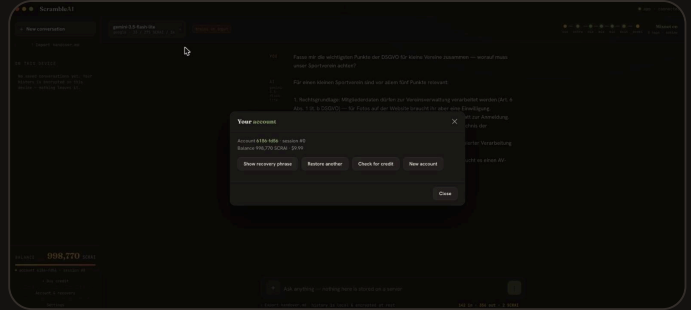
Buying credit. Fixed amounts, three rails: NYM (native · Nyx), Bitcoin (on-chain · LN), Monero (coming soon).



Paying in NYM, natively. \$5 → 500,000 SCRAI on the Nyx chain: purple QR, bare address, and the invoice memo as a first-class field — a transfer without it can't be matched, so the UI treats it that way. Rate locked, window counting down.



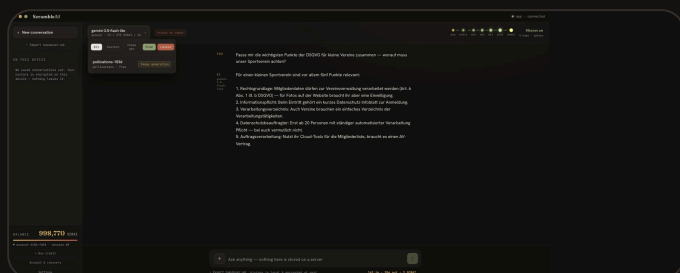
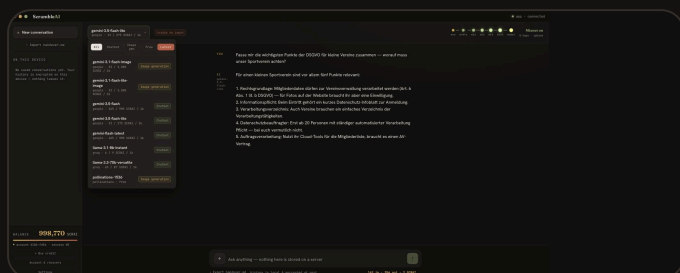
After settlement. The paid credit sits as blind-signed books next to the balance – "500,000 SCRAI held – redeems on your next message". Buy and first spend stay time-decoupled on purpose.



The account. A fingerprint instead of a name; recovery through the phrase alone.

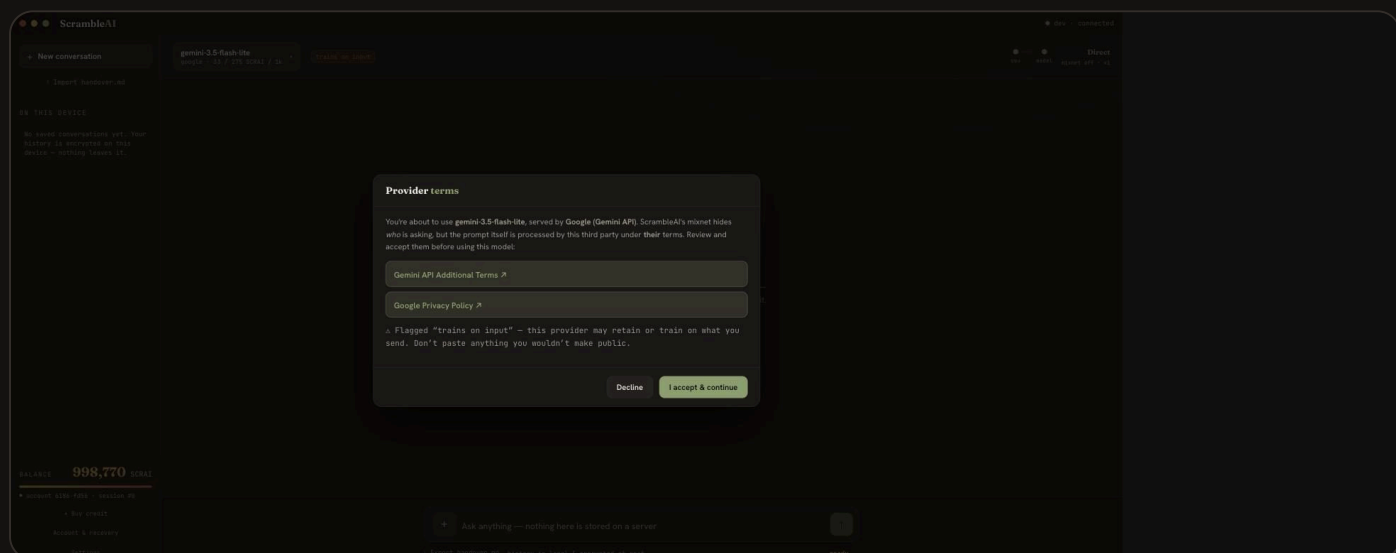
An honest catalog instead of a **black box**

The model catalog comes live from the providers (Google Gemini, Groq — text and image) and is filtered server-side against the price list: the price in the picker is the price on the bill, shown in SCRAI per 1k tokens. Models that train on input carry a visible **trains on input** badge, and before first use the app shows the provider's terms — because the mixnet hides *who* is asking, not *where* the answer is produced.



The catalog. Filters: All • Chatbot • Image gen • Free • Latest — additively combinable.

Free filter. Only models that cost nothing — and are billed exactly that way.



Provider terms. Consent per model, with a plain-language warning about what the provider may do with inputs.

One core, Rust throughout

The entire production path is Rust — one shared crypto core, used byte-compatibly by the desktop client and the server.



scrai-core

Shared library: Coconut e-cash, double-spend quorum, price table, billing math, request signatures. One bug fix lands on both sides.

Desktop app

Tauri 2. A Rust process with the embedded Nym SDK, native OCR and a local NER model; the murmur UI runs as a web frontend in the webview — identical for the coming mobile builds.

scrai-server

A Rust binary acting as a Nym service provider. Persists sessions, quorum and payroll in SQLite; model catalog live from Gemini + Groq; vision uploads in acked chunks.

Hardened

Core paths unit-tested (billing, signatures, e-cash lifecycle, upload reassembly); verified end-to-end over the real mixnet.

From one server to a network nobody can switch off

A clean split: what already runs, and what comes next.

Running today

LIVE

The full anonymous loop

Chat, vision uploads and payments end-to-end over the real mixnet: account-signed purchases (native NYM on Nyx with memo matching via our own chain watcher, plus Bitcoin/Lightning through BTCPay), blind-signed Coconut ticketbooks, metered sessions with per-message reserve → settle, and a paywalled model catalog served live from the providers.

LIVE

Federation-grade cryptography

The Coconut layer is t-of-n from day one — threshold issuance with one aggregated verification key is implemented and tested in the shared core; the current deployment simply runs it as 1-of-1. Scaling to many scrai-servers where **one balance is valid everywhere** is deployment and key ceremony, not a rewrite. Double-spends are accepted offline and caught by a majority quorum; sanctions only on identify() proof.

Next

NEXT

Server discovery that needs no permission

Each server announces itself with a **signed descriptor** — Nym address, federation key, model list, signed price list — in a registry on the Nyx chain (announcing costs a small fee: built-in spam protection), replicated via gossip between federation members. The app loads the directory over the mixnet and verifies every entry against the federation key: **prices are client-verifiable**, so a server can neither slip in per-customer pricing nor silent increases.

NEXT

More rails, bigger replies

Monero as a third payment rail, ticketbooks sized to the purchase tiers, and image generation over SURB budgets for megabyte-sized replies.

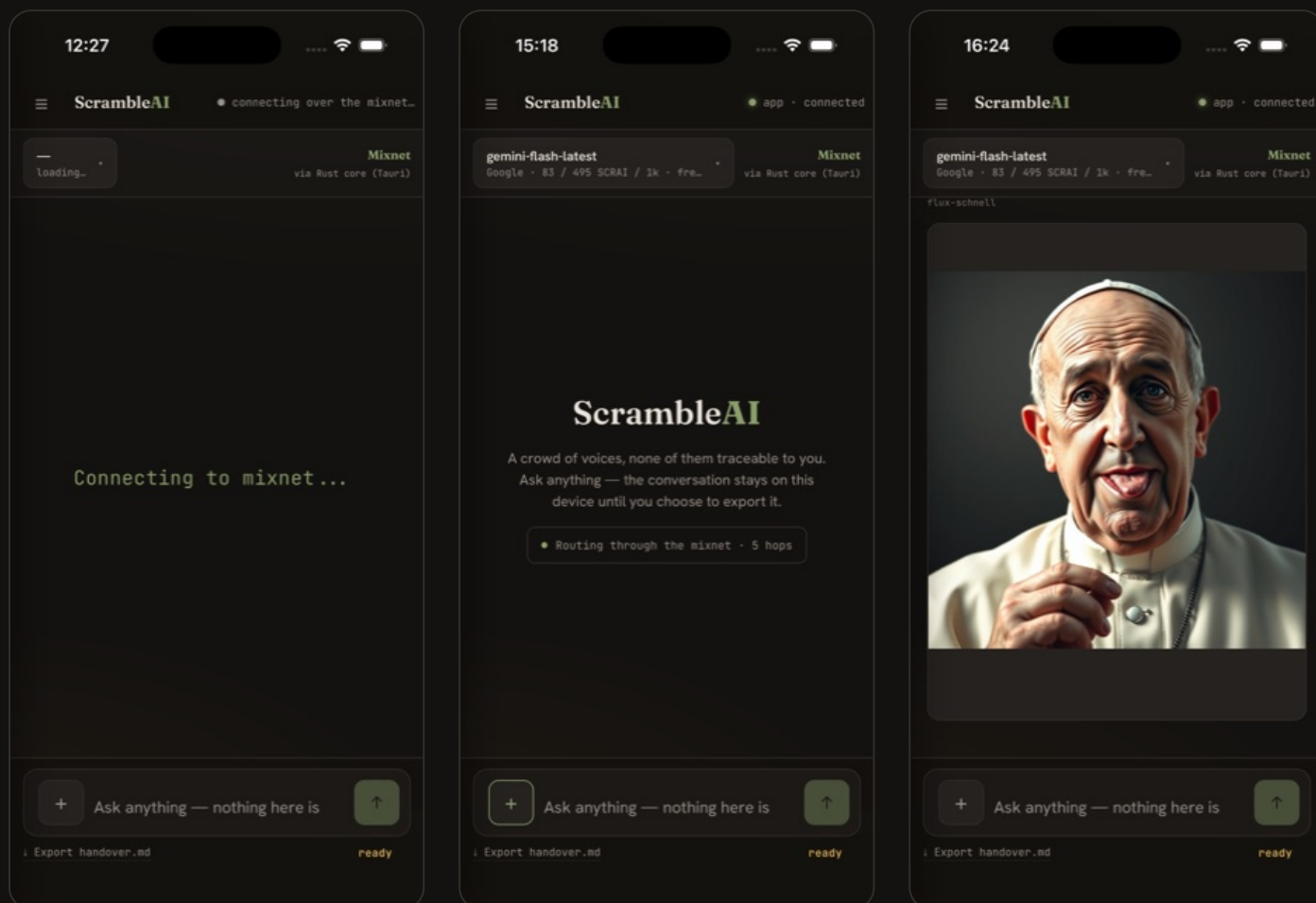
NEXT

Same core, every platform

iOS already runs the full stack — see the chapter below. Android follows via the same Tauri 2 setup with the same Rust core and the same UI; keys move into the OS keychain.

The same app, in your pocket

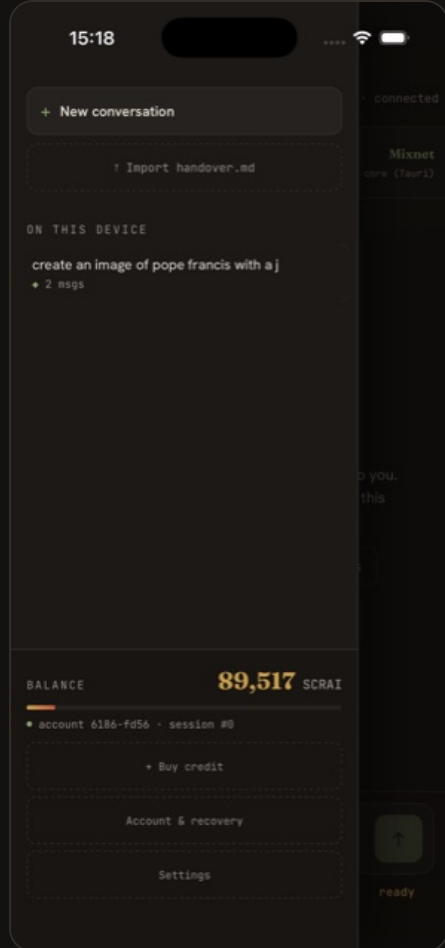
Not a second codebase: the identical Rust core — Nym SDK, Coconut wallet, privacy guard, native OCR — compiled for iOS with Tauri 2, running the same UI. Chat, payments and image generation ride the mixnet from the phone today.



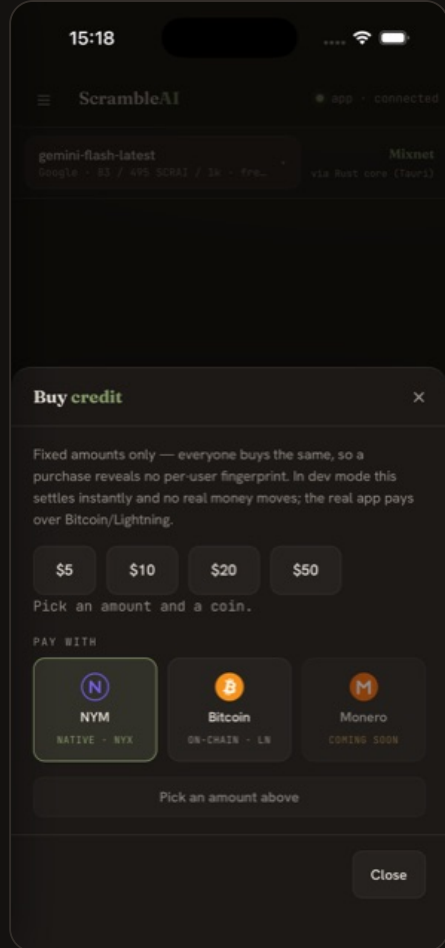
Connecting. The embedded Nym client builds its route on launch.

Ready. Routing through the mixnet • 5 hops.

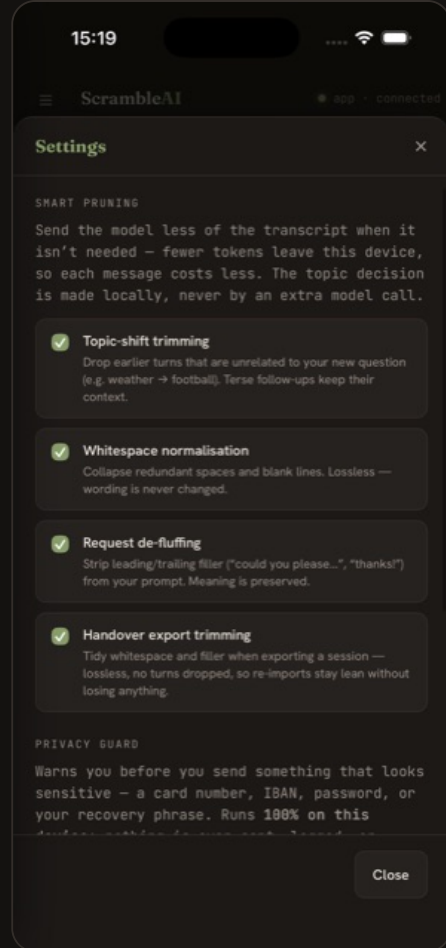
Image generation. flux-schnell, a megabyte reply delivered through SURBs.



On this device. Encrypted local history, the SCRAI balance below.



Buy credit. Fixed tiers as a bottom sheet – NYM native, Bitcoin, Monero next.



Settings. Smart pruning and the on-device privacy guard – identical switches, phone-sized.

ScrambleAI · anonymous AI over the Nym mixnet

Figures: mixnet RTT and pruning savings measured for real · payments currently testnet